ChatGPT

Fixing keyboard RGB control in g-helper-linux

Executive diagnosis

Yes, this is fixable in your project, but not with a small tweak to the existing sysfs keyboard-backlight path. Your repository already has the architectural shape for a real lighting backend: there is a `LightingProvider` trait, a daemon/UI split, a capability-driven API, and a lighting request shape that already anticipates `rgb_hex`. The problem is that Aura support is still missing end-to-end: your own docs say `has_aura` exists in the payload shape but is not currently probed true, the provider matrix says Aura/RGB is not implemented, and the daemon's current `SetLighting` implementation only talks to the sysfs LED backend and explicitly rejects RGB color requests. [1]

The strongest sign that your pasted diagnosis is directionally correct is that the daemon currently holds both an `asusd` platform provider and a sysfs keyboard-backlight provider, but the lighting setter ignores `asusd` entirely and writes only to `KbdBacklightSysfs`. That code path accepts only `Off` and `Static`, and if `rgb_hex` is present it returns the exact "requires asusd/Aura" style error you described. In other words, the failure is not primarily "permissions" or "bad UI state"; it is that the RGB backend is not wired up yet. [2]

A second important point is that the CPU `permission_denied` warning in your diagnostics is a separate issue. Your project's provider docs treat CPU sysfs control and keyboard lighting as different backends, and the current RGB failure path comes from the lighting/backend split, not from the CPU write problem. [3]

What the repository does today

Your current DBus contract documents `SetLighting` as a sysfs-only keyboard-backlight backend. The accepted input map already includes `brightness`, `mode`, and optional `rgb_hex`, but the same doc also says the current backend accepts only `Off` and `Static`, and that RGB is rejected. That is exactly consistent with the daemon implementation in `crates/rog-daemon/src/main.rs`. [4]

The capability model in `rog-core` also shows where the implementation stopped. `DeviceCaps` contains both `has_aura` and `has_kbd_backlight`, and `LightingProvider` exists as a trait, but your docs state that `has_aura` is not currently probed to true and that Aura/RGB support is still planned rather than implemented. The provider matrix says the `asusd` provider currently covers only profiles and charge limit, while the `kbd_backlight` provider is brightness-only with no Aura/RGB support. [5]

There is also a model mismatch that matters for the fix. In `rog-core`, `LightingState` only contains `brightness` and `mode`; it has no color field and no representation of supported modes. But the UI-side `LightingInfo` already anticipates richer state such as `supports_rgb` and `supported_modes`, and `PendingLighting` already has an `rgb_hex` field. That means your repo is half-prepared for Aura, but the shared domain model still lags behind the richer lighting concept that the daemon and UI want to use. [6]

The UI is also not the right place to solve this. Your GUI spec says the UI runs unprivileged and routes hardware-affecting actions through `rog-helperd` on the session bus, and the DBus API doc names that daemon as the source of truth for `SetLighting`. So the correct fix path is provider and daemon work first, with a UI follow-up after the daemon can actually drive Aura. [7]

What upstream asusd already provides

The upstream `asusctl` stack already exposes a dedicated Aura interface on the system bus. In `rog-dbus`, the generated Aura proxy targets service `xyz.ljones.Asusd`, object path `/xyz/ljones/Aura`, and interface `xyz.ljones.Aura`. That proxy exposes writable properties for `brightness`, `led_mode`, `led_mode_data`, and `led_power`, along with capability-style properties such as `supported_basic_modes`, `supported_basic_zones`, and `supported_power_zones`, plus APIs like `all_mode_data` and `direct_addressing_raw`. In practical terms, upstream `asusd` already models the exact backend your project is missing. [8]

Just as important, upstream treats Aura as a separate DBus interface from the platform-profile interface. The platform proxy is `xyz.ljones.Platform` on `/xyz/ljones`, while Aura is `xyz.ljones.Aura` on `/xyz/ljones/Aura`. Upstream also ships object-manager helpers that enumerate interfaces from the system bus and let clients test whether an interface is present at all. That means your `has_aura` detection should not be derived from “platform capabilities” alone; the more robust test is whether the `xyz.ljones.Aura` object/interface is actually exposed by `asusd`. [9]

That separation matters because your pasted diagnosis talks about a “Capabilities property” approach. The deeper upstream picture is better than that: Aura is not just a boolean feature hanging off the platform interface, but a dedicated DBus object with its own methods, properties, and wire types. So the clean fix is to probe for the Aura interface itself. [10]

Upstream also already maintains support metadata for keyboard lighting. The `asusctl` README says keyboard LED support depends on entries in `rog-aura/data/layouts/aura_support.ron`, and the `rog-aura` README explains that support is keyed by DMI `board_name` and that the crate covers RGB keyboards across several ASUS laptop lines, including USB devices and some I2C-connected cases. That means, if your machine is not surfaced by `asusd`, the first suspect is not “your app must implement raw HID immediately,” but rather “upstream model support may be missing or incomplete.” [11]

Why your laptop still shows no RGB control

The most likely explanation, based on the current repository state, is simple: even if `asusd` on your machine already exposes `xyz.ljones.Aura`, your app never probes or uses that interface. Your current startup flow records sysfs LED endpoints, CPU sysfs endpoints, `asusd-platform`, and `supergfxd`, but the capability docs and roadmap explicitly say Aura probing is not done yet. That matches the behavior you reported: `has_profiles` and `has_charge_limit` can be true through `asusd`, while `has_aura` stays false and RGB requests fail. [12]

There is also a very plausible second-layer failure mode: your laptop may not currently be exposed by upstream Aura support, even though the basic keyboard backlight LED exists in sysfs. The `asusctl`

project warns that keyboard LED support depends on `aura_support.ron`, and its README ties that to specific board data. So the situation can look like this: Linux exposes `asus::kbd_backlight` for brightness, but `asusd` does not expose the richer Aura interface for that exact model or firmware combination. That would produce exactly the "brightness works, RGB does not" split that you are seeing. This is an inference, not a live-system confirmation, but it is strongly supported by how upstream `asusctl` documents keyboard LED support. [11]

A third reason the user-facing app still would not feel "fixed" even after backend work is the UI. Your roadmap says "Aura / RGB lighting" is planned and missing, with "UI support beyond current keyboard brightness and mode placeholders" still absent. The UI code backs that up: it has `PendingLighting { brightness, mode, rgb_hex }` and `LightingInfo { supports_rgb, supported_modes, ... }`, but the implemented quick-action handler currently submits `rgb_hex: None`, which means the existing UI path is not yet a real RGB control surface. [13]

So the right answer to "can we fix it" is: yes, probably, but the full fix is two-part. First, add a real Aura backend in the provider/daemon path. Second, add at least a minimal color-and-mode UI on top of that new backend. [14]

The implementation that is most likely to work

The lowest-risk architecture is to add a dedicated `AsusdAuraProvider` under `crates/rog-providers/src/`, wire it into `crates/rog-daemon/src/main.rs`, and keep the current sysfs provider as a fallback for machines that only support brightness. That fits your existing provider pattern exactly: you already have separate providers for `asusd` platform, `supergfxd`, sysfs lighting, and telemetry, and the daemon already talks to system services while the UI remains unprivileged. [15]

The first change should be detection, not UI. Add a system-bus Aura probe that checks for interface presence via object-manager enumeration or a direct proxy build against `xyz.ljones.Asusd` and `/xyz/ljones/Aura`. Upstream's own helper code already enumerates DBus managed objects to discover interfaces, so your implementation can follow that pattern. Once that probe succeeds, set `caps.has_aura = true`, record an `asusd-aura:` endpoint in diagnostics, and stop relying on any platform-capability shortcut for RGB detection. [16]

The second change should be dispatch. Right now `SetLighting` always goes to `KbdBacklightSysfs`. Instead, it should prefer `AsusdAuraProvider` when present, and fall back to sysfs only when Aura is absent. That makes the behavior explicit:

- **Aura available**: accept RGB color and richer modes.
- **Aura absent but sysfs available**: keep current brightness-only behavior.
- **Neither available**: return unsupported.
  That fallback matrix is already aligned with your capability-driven UI philosophy and existing feature-access messaging. [17]

The third change should be your shared lighting model. Today `LightingState` cannot represent a color at all, while the UI and DBus request shape already anticipate `rgb_hex`. The cleanest fix is to split "lighting capabilities/info" from "lighting state/request." For example, keep a small `LightingState` or

`LightingRequest` with `brightness`, `mode`, and optional `rgb_hex`, and add a separate `LightingCapabilities` or similar type that carries `backend`, `device`, `max_brightness`, `supports_rgb`, and `supported_modes`. That would make your `LightingProvider` trait and your DBus/UI payloads line up instead of living in two different design eras. [18]

The fourth change is dependency strategy. Because the upstream Aura interface uses custom types such as `AuraModeNum`, `AuraEffect`, `LedBrightness`, and power-zone enums, the fastest robust path is to reuse upstream Aura wire definitions rather than reverse-engineering the zvariant shapes by hand. The upstream `asusctl` tree already contains `rog-dbus` and `rog-aura` modules that define those concepts. If you do not want an external dependency, you can still mirror only the minimal subset of those types in-tree, but that is more maintenance risk than reusing the already-existing upstream contract. [19]

A practical patch shape would look like this:

```
// new: crates/rog-providers/src/asusd_aura.rs
pub struct AsusdAuraProvider {
    conn: zbus::Connection,
    path: String,
}

impl AsusdAuraProvider {
    pub async fn connect_system() -> RogResult<Option<Self>> {
        // enumerate managed objects on xyz.ljones.Asusd
        // find interface "xyz.ljones.Aura"
        // return provider with detected object path
    }

    pub async fn probe_caps(&self) -> RogResult<LightingCaps> {
        // read supported_basic_modes, supported_brightness, device_type
    }
}

impl LightingProvider for AsusdAuraProvider {
    async fn get_state(&self) -> RogResult<LightingState> {
        // read brightness + led_mode (+ rgb if represented by your model)
    }

    async fn set_state(&self, state: LightingState) -> RogResult<()> {
        // set brightness / led_mode / led_mode_data
    }
}
```

The fifth change is UI follow-through. Do not stop at backend parity, because the current implemented UI path still submits `rgb_hex: None`. Once the daemon can truly handle Aura, add a simple mode selector and color selector in the lighting section, and enable them only when the daemon reports `supports_rgb`

`= true`. Your GUI spec says the UI should degrade gracefully; that is exactly what a backend-flagged color picker would do. [20]

What success and failure will look like

The clearest success condition is live interface visibility. If your machine exposes `xyz.ljones.Aura` on `xyz.ljones.Asusd`, then a provider-backed fix in `g-helper-linux` is the right path. Upstream's generated proxy points to `/xyz/ljones/Aura`, and upstream also provides object-manager enumeration for interfaces. If that interface exists on your system, your app should be able to read/write brightness and built-in Aura modes through DBus rather than through sysfs. [21]

A second good validation step is upstream parity. The `asusctl` manual describes `asusd` as the system daemon exposing a safe DBus interface, and it documents Aura-related CLI behavior such as `asusctl aura -n`. If the upstream Aura path on your machine cannot switch modes either, then adding a provider in your app will not magically solve the hardware support gap. At that point, the likely work moves upstream: confirm your board name, compare against `aura_support.ron`, and only consider raw HID or lower-level experimentation if upstream Aura support really is absent for your model. [22]

A failure condition that should change your plan is this: if `xyz.ljones.Aura` is absent, and upstream `asusctl` also cannot control keyboard RGB, then the problem is probably not in `g-helper-linux` first. The deeper issue is that `asusd` is not surfacing Aura for this device, which may mean unsupported board metadata, a packaging/version mismatch, or genuine lack of an upstream backend for your keyboard path. In that case, the best next technical target is upstream support coverage rather than duplicating `rog-aura` logic in your project. [23]

The shortest path to a real fix, then, is this: treat Aura as a first-class provider in your daemon, detect it by interface presence, route `SetLighting` through it when available, keep sysfs as fallback, and only then add UI controls for color and richer modes. That plan matches your repo's existing architecture, aligns with upstream `asusctl`'s design, and addresses the actual reason RGB is failing today. [24]

---

[1] https://github.com/UdayaSri0/g-helper-linux/blob/main/crates/rog-providers/src/traits.rs
https://github.com/UdayaSri0/g-helper-linux/blob/main/crates/rog-providers/src/traits.rs

[2] [12] [17] g-helper-linux/crates/rog-daemon/src/main.rs at main · UdayaSri0/g-helper-linux · GitHub
https://github.com/UdayaSri0/g-helper-linux/blob/main/crates/rog-daemon/src/main.rs

[3] [15] raw.githubusercontent.com
https://raw.githubusercontent.com/UdayaSri0/g-helper-linux/refs/heads/main/docs/PROVIDER_MATRIX.md

[4] raw.githubusercontent.com
https://raw.githubusercontent.com/UdayaSri0/g-helper-linux/refs/heads/main/docs/DBUS_API.md

[5] [6] [18] g-helper-linux/crates/rog-core/src/model.rs at main · UdayaSri0/g-helper-linux · GitHub
https://github.com/UdayaSri0/g-helper-linux/blob/main/crates/rog-core/src/model.rs

[7] [14] [24] raw.githubusercontent.com
https://raw.githubusercontent.com/UdayaSri0/g-helper-linux/refs/heads/main/docs/GUI_SPEC.md

8 21 raw.githubusercontent.com
https://raw.githubusercontent.com/NeroReflex/asusctl/main/rog-dbus/src/zbus_aura.rs

9 10 raw.githubusercontent.com
https://raw.githubusercontent.com/NeroReflex/asusctl/main/rog-dbus/src/zbus_platform.rs

11 19 23 GitHub - NeroReflex/asusctl: Daemon and tools to control your ASUS ROG laptop. Supersedes rog-core. · GitHub
https://github.com/NeroReflex/asusctl

13 raw.githubusercontent.com
https://raw.githubusercontent.com/UdayaSri0/g-helper-linux/refs/heads/main/docs/ROADMAP.md

16 raw.githubusercontent.com
https://raw.githubusercontent.com/NeroReflex/asusctl/main/rog-dbus/src/lib.rs

20 https://github.com/UdayaSri0/g-helper-linux/blob/main/crates/rog-ui/src/main.rs
https://github.com/UdayaSri0/g-helper-linux/blob/main/crates/rog-ui/src/main.rs

22 raw.githubusercontent.com
https://raw.githubusercontent.com/NeroReflex/asusctl/main/MANUAL.md